B.Tech. BCSE497J - Project-I

CONFIDENCE CALIBRATION AND HALLUCINATION REDUCTION IN SMALL LANGUAGE MODELS

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCT0162 NATALIYA SINGH

23BDS0048 ANISHA DHOOT

23BDS0055 YASH HIRANI

Under the Supervision of

Prof. Swathi J N

Professor, VIT

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to-be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled "Confidence Calibration and Hallucination Reduction in Small Language Models", submitted by us for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT, is a record of bonafide work carried out by us under the supervision of Prof. Swathi J N.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 26/10/26

Signature of the Candidate

CERTIFICATE

This is to certify that the project entitled "Confidence Calibration and Hallucination Reduction in Small Language Models", submitted by Nataliya Singh (Reg. No. 23BCT0162), Anisha Dhoot (Reg. No. 23BDS0048), Yash Hirani (Reg. No. 23BDS0055), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026–2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfils the requirements and regulations of the University and, in my opinion, meets the necessary standards for submission.

Place : Vellore
Date : 26/10/26

Signature of the Guide

Examiner(s)

Sendhil Kumar K S
Sendhil Kumar K S
Head of the Department
B.Tech Computer Science and Engineering

ACKNOWLEDGEMENTS

We are deeply grateful to the management of Vellore Institute of Technology (VIT) for giving us the opportunity and the resources to undertake this project. The support and infrastructure provided by VIT allowed us to explore our ideas and develop them to their fullest potential.

Our sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his support and encouragement. His leadership and vision have inspired us to strive for excellence, and his commitment to academic excellence and innovation has been a constant source of motivation.

We express our profound appreciation to Sendhil Kumar K S, Head of the Department, School of Computer Science and Engineering (Data Science), for his guidance and continuous support. His advice and constructive feedback were valuable in shaping the direction of this project and in overcoming the challenges we met along the way.

We are immensely thankful to our project guide, Prof. Swathi J N, for the dedicated mentorship and detailed feedback we received throughout the project. The guidance on framing the research questions, designing the experiments and presenting the results was essential to the completion of this work and has enriched our academic experience.

We thank everyone who supported us during this project.

Nataliya Singh / Anisha Dhoot / Yash Hirani

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	List of Figures	**vi**
	List of Tables	**vii**
	List of Abbreviations	**viii**
	Symbols and Notations	**viii**
	ABSTRACT	**ix**
1	**INTRODUCTION**	**1**
1.1	Background	1
1.2	Motivations	1
1.3	Scope of the Project	1
1.4	Organisation of the Report	2
2	**PROJECT DESCRIPTION AND GOALS**	**3**
2.1	Literature Review	3
2.1.1	Hallucination in Language Models	3
2.1.2	Confidence Estimation and Calibration	3
2.1.3	Selective Prediction and Abstention	3
2.1.4	Parameter-Efficient Fine-Tuning of SLMs	4
2.1.5	Summary of Key Works	4
2.2	Research Gap	4
2.3	Objectives	5
2.4	Problem Statement	5
2.5	Project Plan	5
3	**TECHNICAL SPECIFICATION**	**7**
3.1	Requirements	7
3.1.1	Functional Requirements	7
3.1.2	Non-Functional Requirements	7
3.2	Feasibility Study	7
3.2.1	Technical Feasibility	7
3.2.2	Economic Feasibility	8
3.2.3	Social Feasibility	8
3.3	System Specification	8
3.3.1	Hardware Specification	8
3.3.2	Software Specification	8
3.4	Background Concepts	8
4	**DESIGN APPROACH AND DETAILS**	**10**
4.1	System Architecture	10
4.2	Design	10
4.2.1	Data Flow Diagram	10
4.2.2	Use Case Diagram	11
4.2.3	Sequence Diagram	11
4.3	Problem Formulation	12

Sl.No	Contents	Page No.
4.3.1	Task Definition	12
4.3.2	Hallucination and Correctness	12
4.3.3	Calibration Objective	13
5	**METHODOLOGY AND TESTING**	**14**
5.1	Module Description	14
5.1.1	Module 1: Data Curation and Dataset Construction	14
5.1.2	Module 2: Data Preparation and Controlled Abstention Mixing	14
5.1.3	Module 3: QLoRA Fine-Tuning of Phi-3-mini	15
5.1.4	Sequence-Level Confidence Estimation	16
5.1.5	Modules 4 and 5: Evaluation, Analysis and Figures	16
5.1.6	Module 6: Demonstration Chatbot	16
5.2	Testing and Validation	17
5.2.1	Data and Pipeline Tests	17
5.2.2	Metric Definitions	17
5.2.3	Experimental Setup	17
6	**PROJECT DEMONSTRATION**	**18**
6.1	The Demonstration Website	18
6.2	Example Interaction	18
6.3	Running the Demonstration	19
7	**RESULT AND DISCUSSION**	**20**
7.1	Hallucination Reduction	20
7.2	Calibration Improvement	21
7.3	Confidence Separation	22
7.4	Cost of Abstention: False Refusals	23
7.5	Contribution of Each Training Stage	23
7.6	Qualitative Error Analysis	24
7.7	Answers to the Research Questions	24
7.8	Cost Analysis	24
7.9	Discussion and Limitations	25
8	**CONCLUSION AND FUTURE WORK**	**26**
8.1	Conclusion	26
8.2	Future Work	26
9	**REFERENCES**	**27**
A	**KEY CODE EXCERPTS**	**29**
A.1	Controlled abstention mixing (prepare_expc_data.py)	29
A.2	4-bit and LoRA configuration (train_lora.py)	29
A.3	Confidence and refusal-aware correctness (eval_compare_fixed.py)	29
A.4	Expected Calibration Error (eval_compare_fixed.py)	29

List of Figures

Figure No.	Title	Page No.
1	Project plan: tasks and weeks.	6
2	Framework of the proposed refusal-aware calibration study.	10
3	Level-1 data flow diagram of the evaluation pipeline.	11
4	Use case diagram of the calibration and abstention system.	11
5	Sequence of one chat query.	12
6	Home page of the demonstration website.	18
7	Chat widget answering one question and abstaining on another.	19
8	Refusal behaviour of Base, Normal FT and IDK FT.	20
9	False refusals (over-caution) and missed abstentions (hallucinations) per system.	21
10	Reliability diagram for Base, Normal FT and IDK FT (10 equal-width bins); the dashed line is perfect calibration.	22
11	Confidence on correct and incorrect responses for each system.	23
12	ROC curves for separating correct from incorrect responses using sequence-level confidence.	23

List of Tables

Table No.	Title	Page No.
1	Summary of key related work.	4
2	Software used in the project.	8
3	Outcome matrix used for hallucination and refusal metrics.	12
4	Project modules and the files that implement them.	14
5	Partition sizes for the two training mixtures (abstention examples in parentheses).	15
6	Training configuration.	15
7	Tests and validation checks.	17
8	Hallucinations and refusals on the IDK-15 test set (79 unanswerable, 370 answerable).	20
9	Calibration and answer quality on the IDK-15 test set (refusal-aware correctness, token-F1 threshold 0.5).	21
10	Changes attributable to each stage.	24
11	Cost and benefit of abstention training.	25

List of Abbreviations

AUROC	Area Under the Receiver Operating Characteristic Curve
Brier	Brier Score (mean squared error between confidence and correctness)
ECE	Expected Calibration Error
FN	False Negative (missed abstention)
FP	False Positive (false refusal)
FRR	False Refusal Rate
FT	Fine-Tuning
HR	Hallucination Rate
IDK	I Don't Know (abstention)
LLM	Large Language Model
LoRA	Low-Rank Adaptation
NF4	4-bit NormalFloat
NLL	Negative Log-Likelihood
PEFT	Parameter-Efficient Fine-Tuning
QA	Question Answering
QLoRA	Quantised Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
ROC	Receiver Operating Characteristic
SFT	Supervised Fine-Tuning
SLM	Small Language Model
TP / TN	True Positive / True Negative
VIT	Vellore Institute of Technology
VRAM	Video Random Access Memory

Symbols and Notations

ρ	Target abstention ratio in the training mixture (0.15 for IDK FT)
τ	Token-F1 threshold for correctness (0.5)
θ	Trainable LoRA adapter parameters
q, r, r*	Query, model response and reference answer
z(q)	Answerability label (1 if the query is unanswerable)
a(r)	Abstention indicator (1 if the response is a refusal)
c(q)	Sequence-level confidence in [0, 1]
y(q, r)	Refusal-aware correctness label (0 or 1)
N_reg	Size of the answerable pool (2,537)

ABSTRACT

Small Language Models (SLMs) in the 1B-4B parameter range are cheap to run on-premise and are an attractive base for institution-specific assistants such as university chatbots. Their limited capacity, however, means they often produce incorrect answers with high confidence, which is especially harmful when students rely on the answer for credits, course codes or procedures. This project investigates whether the composition of the fine-tuning data alone can improve confidence calibration and reduce hallucination in an SLM, without any calibration loss, temperature scaling or retrieval.

We fine-tune Phi-3-mini-4k-instruct (3.8 billion parameters) with QLoRA (4-bit NF4, LoRA rank 8) on a question-answer dataset built from public Vellore Institute of Technology (VIT) documents, using a single 4 GB laptop GPU. Three systems are compared on the same 449-query test set (370 answerable, 79 unanswerable): the base model, a conventionally fine-tuned model trained only on answerable questions (Normal FT, 0% abstention data), and a refusal-aware model trained with a controlled 15% share of "I don't know" examples that all use one canonical refusal sentence (IDK FT). Confidence is the length-normalised sequence probability of the generated answer; correctness is refusal-aware, so refusing is right for unanswerable questions and wrong for answerable ones.

Domain fine-tuning roughly doubled answerable accuracy (0.227 to 0.451) but removed all abstention: Normal FT answered all 79 unanswerable questions (hallucination rate 1.000, against 0.506 for the base model). Adding 15% abstention examples cut the hallucination rate to 0.139, gave an IDK F1 of 0.840, lowered ECE from 0.622 (base) to 0.408 (Normal FT) to 0.304 (IDK FT), lowered the Brier score from 0.480 to 0.351 to 0.285 and raised AUROC from 0.557 to 0.764 to 0.810. The cost was 15 false refusals out of 370 answerable queries (4.1%) and a 4.3 percentage-point fall in answerable accuracy. All systems remain over-confident, part of the calibration gain comes from confidently repeating the fixed refusal sentence, and the study uses one model, one abstention ratio, one seed and one institution. The results show that abstention does not emerge from standard fine-tuning and must be explicitly supervised.

Keywords: Small Language Models, Confidence Calibration, Hallucination, R-Tuning, Refusal-Aware Fine-Tuning, QLoRA

CHAPTER 1
INTRODUCTION

1.1 Background

Small Language Models (SLMs) in the 1B-4B parameter range are efficient and can run on-premise at low inference cost. This makes them a natural choice for institution-specific assistants such as a university chatbot. Their small capacity, however, means they frequently generate incorrect answers with high confidence. When students rely on an assistant for credit structures, course codes or procedures, a confidently wrong answer is considerably worse than an honest "I don't know".

Calibration and hallucination are closely related but usually studied separately. A confidence score is only useful if it is calibrated, meaning that answers given with 80% confidence are right about 80% of the time. A well-calibrated model can use its uncertainty to abstain and turn a would-be hallucination into an honest refusal. A poorly calibrated model is just as sure when it is wrong as when it is right, so neither the user nor the system has a signal to catch the fabricated answer.

1.2 Motivations

Domain fine-tuning is the standard way to teach an SLM institution-specific facts. It is not obvious what this does to the model's willingness to say "I don't know": the fine-tuning data contains only answerable questions, so the model may learn that every question in the domain has an answer. Retrieval-augmented generation does not help for questions with nothing to retrieve, such as a student's personal record or real-time information. This project therefore asks whether the composition of the fine-tuning data alone, with a controlled share of explicit refusal examples, can make a small model both better calibrated and less prone to hallucinate.

1.3 Scope of the Project

- One base model (Phi-3-mini-4k-instruct), one institution (VIT), one abstention ratio (15%) compared with a 0% baseline, one random seed (42), and closed-book question answering.
- A VIT question-answer dataset of 2,349 knowledge pairs plus 980 supplementary pairs, with five categories of unanswerable query.
- A controlled-ratio abstention data builder, a QLoRA training script that fits in 4 GB of VRAM, and an evaluation suite covering refusal behaviour (hallucination rate, IDK precision, recall, F1, false refusals) and calibration (ECE, Brier, NLL, AUROC).
- An empirical comparison of Base, Normal FT and IDK FT, with a stage-by-stage ablation and a qualitative error analysis.
- A demonstration web chatbot that shows the answer, the confidence and an explicit abstention.

The project does not include retrieval from the documents at inference time, post-hoc calibration such as temperature scaling, or experiments with other base models; these are listed as future work in Chapter 8.

1.4 Organisation of the Report

Chapter 2 describes the project goals: the literature review, the research gap, the objectives, the problem statement and the project plan. Chapter 3 gives the technical specification: requirements, feasibility and the system specification, together with the background concepts needed to read the results. Chapter 4 presents the design: the architecture, the data flow, use case and sequence diagrams, and the formal problem formulation. Chapter 5 explains the methodology, module by module, and the testing carried out. Chapter 6 demonstrates the working system. Chapter 7 reports and discusses the results. Chapter 8 concludes and lists future work. References are in Chapter 9, and Appendix A shows key code excerpts.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

2.1.1 Hallucination in Language Models

A hallucination is a fluent model output that is unsupported by the input or inconsistent with the facts. Ji et al. [19] survey hallucination in natural language generation and Huang et al. [18] separate factuality from faithfulness hallucinations in LLMs. For an institutional assistant the distinction matters little: either the model knows the fact from training or it does not. Kalai and Vempala [23] show that a calibrated language model must hallucinate on some facts, and Kalai et al. [22] argue that benchmarks which reward guessing and give nothing for abstaining push models to guess under uncertainty. Gekhman et al. [12] find that fine-tuning on new knowledge encourages hallucination, and Porwal et al. [38] observe that domain-adapted models work when test prompts resemble training prompts but invent answers about novel facts. This motivates fine-tuning as the natural baseline for the present study.

Current mitigations are retrieval augmentation [28] and post-hoc detection such as SelfCheckGPT [32]. Retrieval cannot help when there is nothing to retrieve (personal records, real-time data), and post-hoc detection only flags a hallucination after it has been generated.

2.1.2 Confidence Estimation and Calibration

A model is calibrated when its stated confidence matches its accuracy. Guo et al. [14] show that modern neural networks are miscalibrated and popularise Expected Calibration Error [34] and temperature scaling; the Brier score [4] is a binning-free alternative. For generative models the confidence must be derived from a sequence of tokens. Malinin and Gales [31] support the length-normalised sequence likelihood as an uncertainty measure. Jiang et al. [20] find T5, BART and GPT-2 poorly calibrated on QA, even after fine-tuning and post-hoc calibration, while Kadavath et al. [21] report good calibration for large models on multiple-choice questions. Other approaches use semantic variance across samples [26] or verbalised confidence [41, 45]. Calibration can also be improved by training: Kapoor et al. [25] show that fine-tuning on a small graded set, including with LoRA, gives better uncertainty than prompting, and Hu et al. [17] find that PEFT improves the AUROC of confidence- and entropy-based hallucination detectors.

2.1.3 Selective Prediction and Abstention

Abstention goes back to Chow's error-reject trade-off [6]. Selective classification [9] couples a predictor with a rejection function and was extended to deep networks by Geifman and El-Yaniv [11]. SQuAD 2.0 [39] introduced unanswerable questions, and Kamath et al. [24] train a calibrator for abstention under domain shift. Recent work puts abstention into the model during training: R-Tuning [48] fine-tunes models to say "I don't know" on questions they get wrong, Cheng et al. [5] build model-specific IDK datasets and compare SFT with preference optimisation, Yang et al. [46] frame it as alignment for honesty, and Feng et al. [10] find knowledge gaps through multi-LLM collaboration. Wen et al. [42] survey the area and list over-abstention as a key open problem.

2.1.4 Parameter-Efficient Fine-Tuning of SLMs

Parameter-efficient methods train a small set of added parameters on a frozen base model: bottleneck adapters [15], prefix tuning [29], prompt tuning [27] and LoRA [16]. LoRA adds low-rank updates to weight matrices and adds no inference latency. QLoRA [8] stores the frozen base in 4-bit NormalFloat with double quantisation and paged optimisers, and together with 8-bit optimiser states [7] it makes the 4 GB training regime used here possible. Biderman et al. [2] show that LoRA learns less than full fine-tuning but forgets less. Phi-3-mini [1] has 3.8 billion parameters and benchmark performance comparable to much larger models [13], which is why it was chosen.

2.1.5 Summary of Key Works

Table 1 summarises the eight works most relevant to this study, with the objective, method, strengths and weaknesses, and main finding of each.

Table 1. Summary of key related work.

Paper	Objective	Methodology	Pros / Cons	Findings
Zhang et al., R-Tuning, NAACL 2024 [48]	Teach LLMs to say "I don't know" for questions beyond their knowledge	Find questions the model gets wrong, build refusal-aware data, fine-tune	Pro: refusal ability transfers to new tasks. Con: large models; abstention share set by data	Refusal behaviour can be learned and generalises
Cheng et al., ICML 2024 [5]	Can assistants know what they do not know?	Model-specific IDK datasets; compare SFT and preference optimisation	Pro: systematic comparison of training styles. Con: 7B+ models, server hardware	Assistants can be trained to recognise knowledge limits
Guo et al., ICML 2017 [14]	Measure calibration of modern neural networks	ECE and reliability diagrams; temperature scaling	Pro: standard metrics and a simple fix. Con: classifiers, not free-form generation	Modern networks are miscalibrated
Jiang et al., TACL 2021 [20]	Calibration of language models on QA	Evaluate T5, BART and GPT-2 confidences, fine-tuning and post-hoc calibration	Pro: early QA calibration study. Con: older models	Models are poorly calibrated; fine-tuning and post-hoc fixes help little
Kapoor et al., NeurIPS 2024 [25]	Teach LLMs to know what they don't know	Fine-tune on a small graded set, including LoRA	Pro: training beats prompting for calibration. Con: needs graded examples	Small fine-tuning sets give better-calibrated uncertainty
Gekhman et al., EMNLP 2024 [12]	Does fine-tuning on new knowledge cause hallucination?	Controlled fine-tuning with known and unknown facts	Pro: explains why domain fine-tuning is risky. Con: does not give a mitigation	New facts are learned slowly and raise hallucination
Hu et al., arXiv 2026 [17]	Effect of PEFT on hallucination detection	Apply confidence and entropy detectors to PEFT models	Pro: links PEFT and detectability. Con: no abstention training	PEFT raises AUROC of unsupervised detectors
Wen et al., TACL 2025 [42]	Survey of abstention in LLMs	Taxonomy of when and how models abstain	Pro: names over-abstention as an open problem. Con: survey only	Over-abstention (refusing known questions) needs direct measurement

2.2 Research Gap

- Abstention work (R-Tuning, Cheng et al.) targets 7B-70B models on server GPUs; sub-4B models on a 4 GB consumer GPU are not studied.
- The abstention share in earlier work is set by how much data happens to exist, not controlled as an experimental variable.
- Calibration and hallucination are usually studied separately; here both are measured on one test set with one refusal-aware correctness definition.

- Over-abstention is named as an open problem [42] but rarely quantified; here the false refusal rate is reported directly.
- Domain fine-tuning is the standard SLM baseline, yet its effect on abstention is rarely measured against a base model and an abstention-aware model.

The present study differs from R-Tuning [48] and Cheng et al. [5] in three ways: the abstention share is explicitly controlled; the model is a sub-4B SLM trained in 4-bit on a consumer 4 GB GPU; and hallucination, calibration and the false-refusal cost are evaluated together in one domain.

2.3 Objectives

Aim. To study whether controlled abstention supervision during parameter-efficient fine-tuning improves confidence calibration and reduces hallucination in a small language model deployed for a domain-specific task. The objectives are:

1. Measure how domain fine-tuning changes the calibration and hallucination behaviour of Phi-3-mini on VIT questions (Base versus Normal FT).
2. Test whether a controlled 15% share of abstention examples reduces the hallucination rate (Normal FT versus IDK FT).
3. Test whether the same change improves calibration (ECE, Brier, reliability diagrams) and discrimination (AUROC) with no calibration loss or temperature scaling.
4. Quantify the cost: false refusals and the drop in answerable accuracy.
5. Run the whole pipeline on a single 4 GB laptop GPU with QLoRA, and package the code, data and results so every number can be reproduced.

These objectives answer three research questions:

1. Does supervised abstention reduce hallucination compared with a model fine-tuned only on answerable data?
2. Does it improve confidence calibration (ECE, Brier score, reliability diagrams) rather than merely making the model hedge indiscriminately?
3. What is the cost to answerable questions, that is, how many questions does the model refuse that it could have answered?

2.4 Problem Statement

Given a closed-book question answering assistant for VIT built on Phi-3-mini, decide for each query whether it can be answered from the institution's documents; answer it correctly if so, abstain if not, and attach a confidence score that reflects the probability of being correct. The model must make the answerability decision itself, using only the question, because nothing is retrieved at inference time. The formal definition is given in Section 4.3.

2.5 Project Plan

The project was planned over sixteen weeks in six tasks, shown in Figure 1: literature survey and problem formulation (weeks 1-3), dataset curation (weeks 4-5), model setup and base evaluation (weeks 6-7), development and fine-tuning (weeks 8-10), testing and evaluation (weeks 11-13), and report and documentation (weeks 14-16).

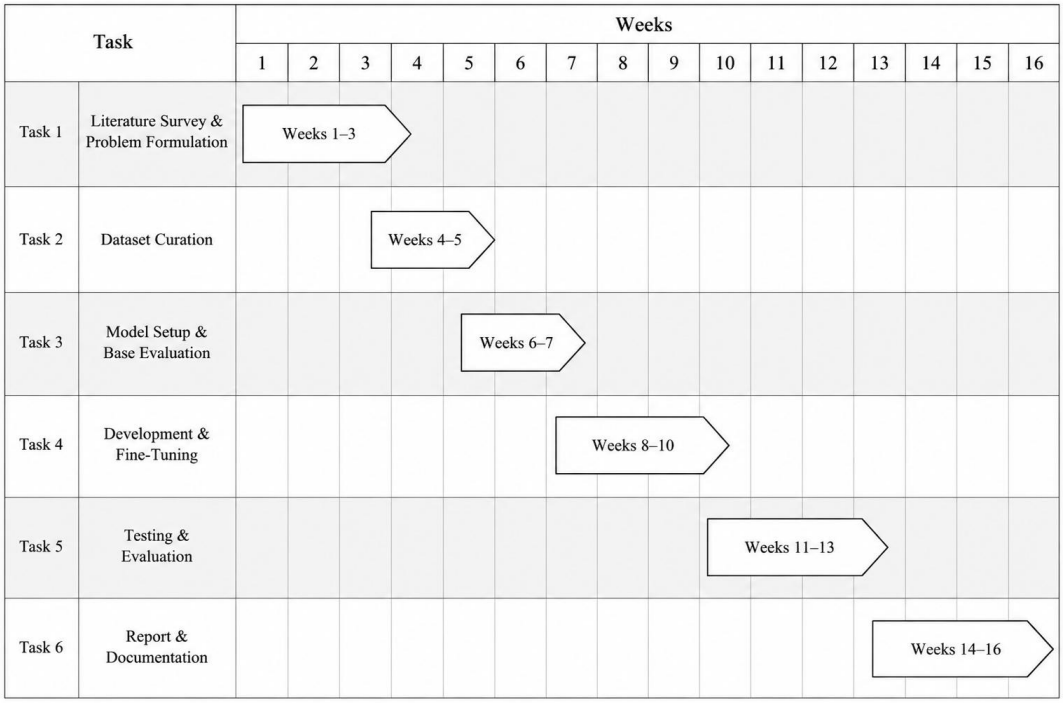


Figure 1. Project plan: tasks and weeks.

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

1. FR1 Dataset builder: read the two QA files, separate answerable and abstention rows, normalise refusals to one canonical string and write train, validation and test files for a chosen abstention ratio.
2. FR2 Dataset verification: confirm there is no train/test leakage and that every abstention row is canonical.
3. FR3 Fine-tuning: train a LoRA adapter on a chosen training file within 4 GB of VRAM.
4. FR4 Generation with confidence: for each test query produce a greedy answer and its length-normalised confidence.
5. FR5 Refusal analysis: classify each response as answer or refusal and compute the hallucination rate, IDK precision, recall and F1, false refusals and answerable accuracy.
6. FR6 Calibration analysis: compute ECE, Brier score, NLL and AUROC, and export per-example confidence and correctness.
7. FR7 Visualisation: produce reliability diagrams, ROC curves, confidence histograms, and refusal and error charts.
8. FR8 Demonstration: a web chatbot that returns an answer, a confidence value and a clear abstention message.

3.1.2 Non-Functional Requirements

- Memory: training and inference fit on one 4 GB laptop GPU (4-bit base, rank-8 adapters, batch size 1 with gradient accumulation, gradient checkpointing).
- Reproducibility: seed 42 for every random operation, and greedy decoding so that predictions are deterministic.
- Fairness: Base, Normal FT and IDK FT share base weights, quantisation, prompt format and decoding settings.
- Modularity: each stage is a separate script, and per-example results are exported so that figures can be rebuilt without a GPU.
- Transparency: the demo always shows which backend produced an answer and never presents a lookup as the fine-tuned model.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The work needs a 3.8 billion-parameter model to be fine-tuned and evaluated on a laptop GPU with 4 GB of memory. This is made feasible by 4-bit NormalFloat quantisation, rank-8 LoRA adapters, a batch size of one with eight gradient-accumulation steps, gradient checkpointing and a paged 8-bit optimiser. Because two 4-bit copies of the model do not fit at once, the systems are

evaluated one after another. All stages of the experiment were carried out within these limits, which demonstrates that the approach is technically feasible on consumer hardware.

3.2.2 Economic Feasibility

The project uses an openly downloadable base model and open-source libraries (PyTorch, Hugging Face transformers, peft, trl, bitsandbytes, scikit-learn, Flask). No paid cloud compute or paid API was needed for the experiments reported here; the only hardware used was the team's existing laptop GPU. The data come from public VIT documents. The extra cost of abstention training is small: the IDK-15 training file has 313 more rows than the IDK-0 file (2,088 against 1,775, about 18% more examples) and uses the same training schedule.

3.2.3 Social Feasibility

A reliable campus assistant benefits students, who often need quick answers about credits, courses and rules. The risk is a confidently wrong answer, which this project aims to reduce by teaching the model to refuse. Only public documents were used, and question rows that relied on non-public information (for example company-specific placement details) were removed before use. The demonstration site states that it is a student project, that answers can be wrong, and that important information must be confirmed on the official VIT website.

3.3 System Specification

3.3.1 Hardware Specification

Training and evaluation ran on a laptop with an NVIDIA RTX 3050 Ti Laptop GPU with 4 GB of VRAM. Because two 4-bit versions of Phi-3-mini do not fit in this memory at the same time, each system is loaded, evaluated over the whole test set and unloaded, and the CUDA cache is cleared before the next one is loaded.

3.3.2 Software Specification

Table 2 lists the software used.

Table 2. Software used in the project.

Component	Purpose
Python, PyTorch [36]	Model execution and training
Hugging Face transformers [44]	Loading Phi-3-mini and generating text
peft [33]	LoRA adapters
trl [43] (SFTTrainer)	Supervised fine-tuning loop
bitsandbytes [7, 8]	4-bit quantisation and paged 8-bit optimiser
datasets, pandas	Data handling
scikit-learn [37]	AUROC and ROC curves
matplotlib	Figures
Flask	Demonstration web chatbot

3.4 Background Concepts

This section explains in plain terms the main quantities used in the rest of the report, so that the results can be read without further background.

- **Calibration.** A model is calibrated when its confidence matches how often it is right. If the model attaches a confidence of 0.8 to a hundred answers, about eighty of them should be correct. A model that is usually more confident than it is accurate is called over-confident.
- **Expected Calibration Error (ECE).** The confidence range is split into ten equal bins. For each bin the gap between the average confidence and the actual accuracy is taken, and the gaps are averaged with weights proportional to the number of answers in each bin. Lower is better; zero is perfect.
- **Brier score.** The mean squared difference between the confidence and the correctness label (0 or 1). It needs no bins and rewards confidence that is both calibrated and sharp. Lower is better.
- **AUROC.** The probability that a randomly chosen correct response has a higher confidence than a randomly chosen incorrect one. A value of 0.5 means the confidence carries no information; 1.0 means it separates the two perfectly.
- **Negative log-likelihood (NLL).** How surprised the model is by the reference answer, measured with the true answer fed to the model token by token. Lower means the model finds the reference more likely.
- **Token F1 and exact match.** Token F1 measures the word overlap between a generated answer and the reference; exact match requires the two to be identical after normalisation. Free-form answers are rarely identical to the reference, which is why token F1 is used for correctness.
- **Abstention.** The model declines to answer. In this work the training target for abstention is one fixed sentence, which makes refusals easy to detect and count.
- **LoRA and QLoRA.** Instead of updating all the weights of a model, LoRA trains small low-rank matrices added to selected layers. QLoRA additionally stores the frozen base model in 4-bit precision, which is what allows a 3.8 billion-parameter model to be trained on a 4 GB GPU.

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 System Architecture

The framework has four layers (Figure 2). The data layer splits the QA corpus into answerable and unanswerable queries. The training layer produces the three systems that are compared: the Base model, which has no adapter, Normal FT, trained on answerable data only, and IDK FT, trained with a controlled share of refusal examples. The inference layer returns an answer with a confidence score, or an "I don't know" response. The evaluation layer computes accuracy, token F1, ECE, Brier score, NLL, AUROC and reliability diagrams.

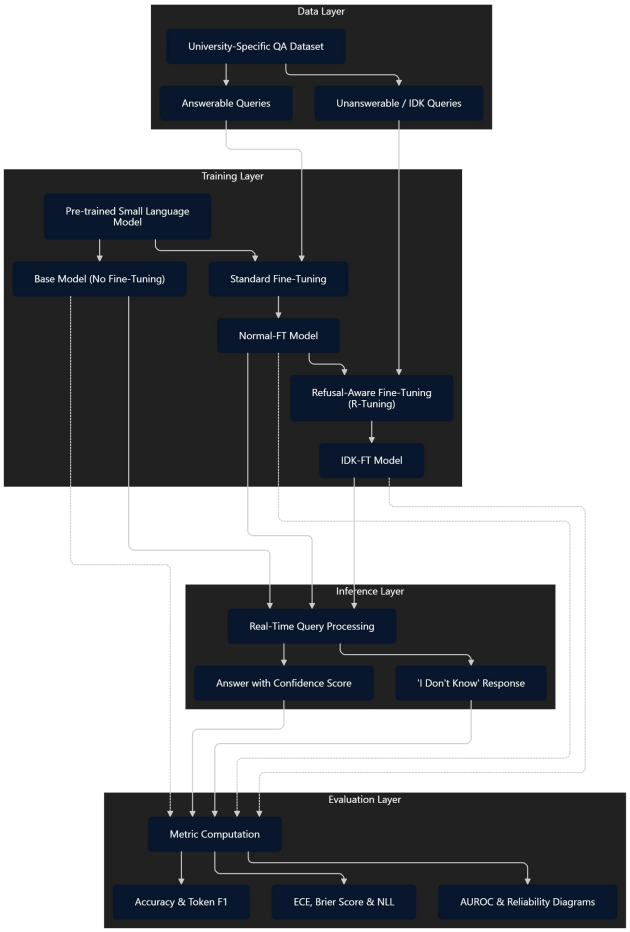


Figure 2. Framework of the proposed refusal-aware calibration study.

4.2 Design

4.2.1 Data Flow Diagram

Figure 3 shows the data flow of the evaluation pipeline. Process 1.0 reads the QA source files (D1) and writes the train, validation and test splits (D2) for the chosen abstention ratio. Process 2.0 reads the training file and writes a LoRA adapter (D3). Process 3.0 loads an adapter, or the plain base model, reads the test file and produces an answer and a confidence for every query. Process 4.0 turns these into refusal and calibration metrics and stores the per-example results (D4). Process 5.0 reads D4 and produces the figures and tables returned to the researcher.

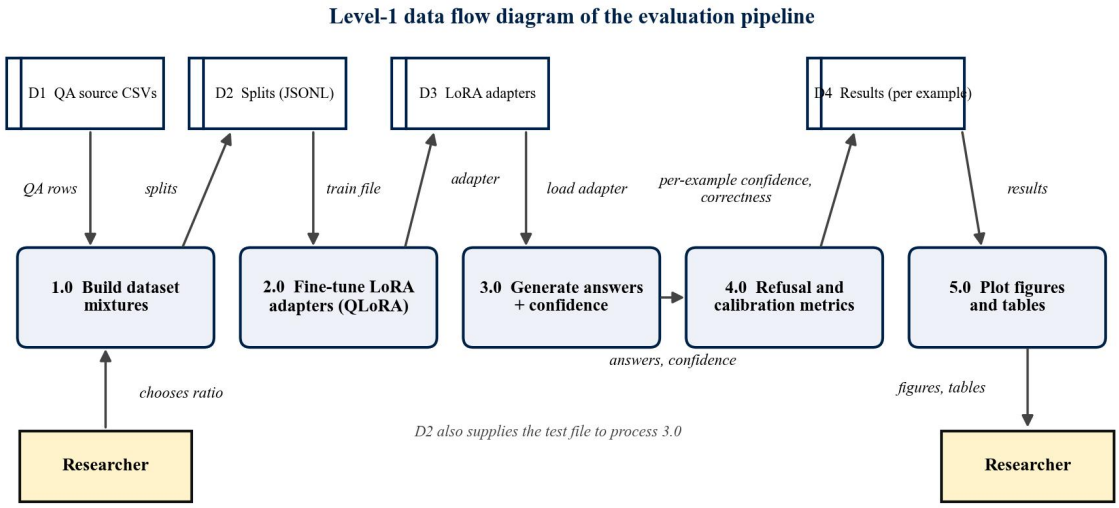


Figure 3. Level-1 data flow diagram of the evaluation pipeline.

4.2.2 Use Case Diagram

Figure 4 shows the two kinds of user. The researcher builds dataset mixtures, trains the adapters, evaluates refusal and calibration, and views the plots and tables. The student or other user asks a question in the demo chatbot and sees the answer together with its confidence, or an explicit abstention.

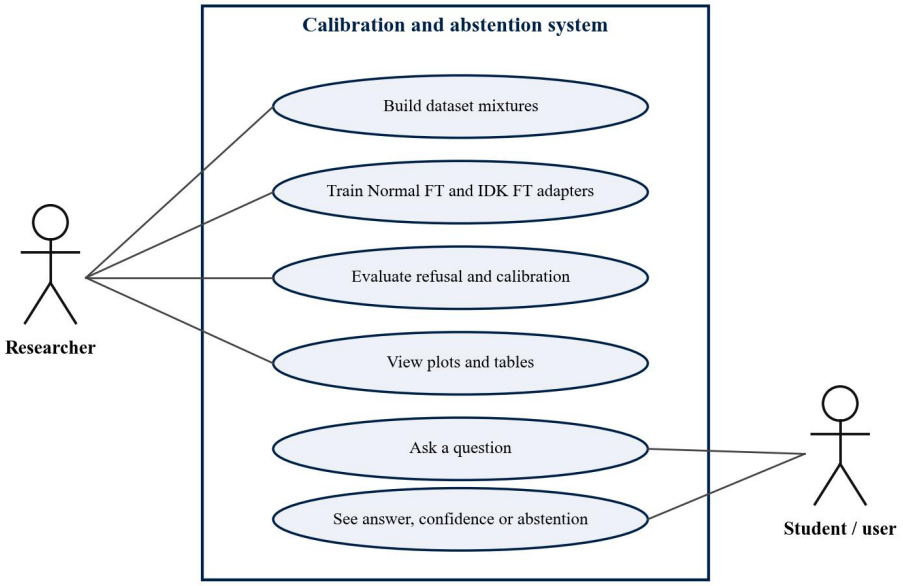


Figure 4. Use case diagram of the calibration and abstention system.

4.2.3 Sequence Diagram

Figure 5 shows what happens when a student sends one question. The browser posts the question to the Flask API, which asks the answer backend. In lookup mode the backend returns the closest stored answer or the canonical refusal. In LoRA mode it generates the answer with Phi-3-mini and the adapter and reports the mean token probability as confidence. The page then shows the answer with a confidence bar, or an abstention.

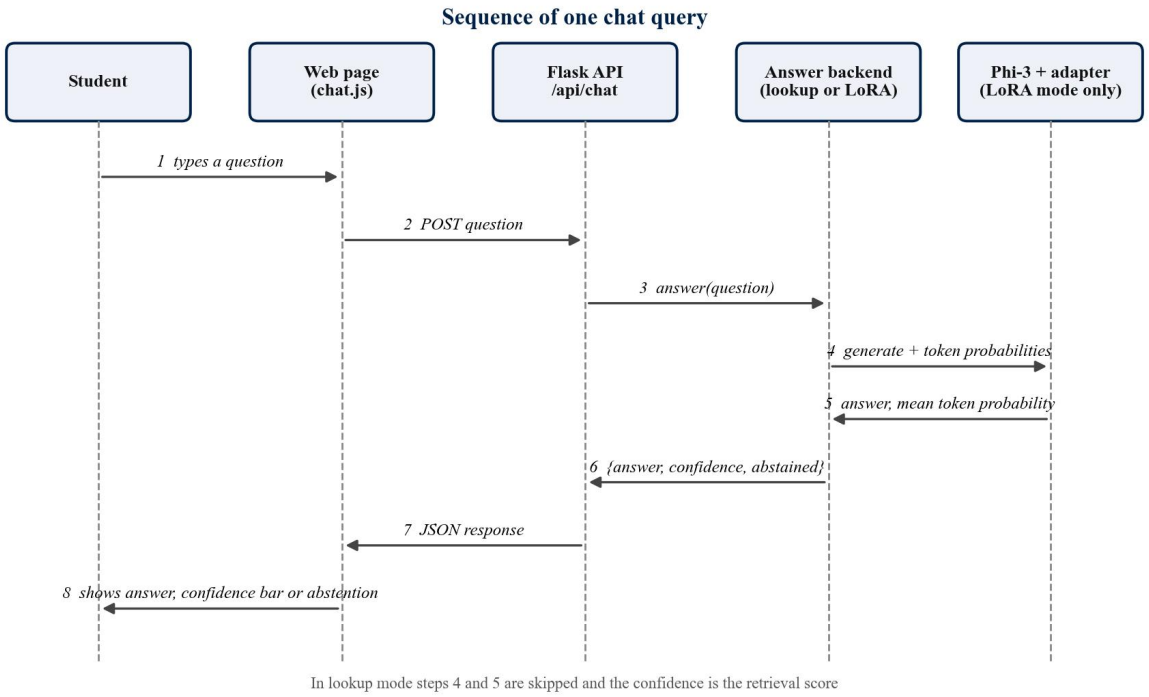


Figure 5. Sequence of one chat query.

The system is implemented as a set of scripts and one small web application, so a class diagram is not drawn. The two classes of the demonstration application are RetrievalBot (dataset lookup) and LoraBot (fine-tuned model).

4.3 Problem Formulation

4.3.1 Task Definition

The task is closed-book question answering for a single institution. Let D be the VIT curriculum and regulation documents. Every query q is either answerable ($q \in Q_A$, its answer can be derived from D and has a reference r*) or unanswerable ($q \in Q_U$), with $Q = Q_A \cup Q_U$ and $Q_A \cap Q_U = \varnothing$. The answerability label is $z(q) = 1[q \in Q_U]$. A question about general knowledge counts as unanswerable even if the model happens to know the answer, because for a student an unverifiable answer looks the same as a fabricated one.

The model returns a response r and a confidence $c(q) \in [0, 1]$. The indicator $a(r) = 1$ means the response is an abstention. Training uses one fixed abstention string; at evaluation a(r) is decided by a lexical refusal detector because the model's output cannot be assumed to match the string exactly. The model must decide answerability itself, since z(q) is unknown to it.

4.3.2 Hallucination and Correctness

Crossing answerability with the model's decision gives four outcomes, shown in Table 3. These outcomes are the basis of every refusal metric used in this report.

Table 3. Outcome matrix used for hallucination and refusal metrics.

	Model abstains: a(r) = 1	**Model answers: a(r) = 0**
Unanswerable (Q_U)	Correct abstention (TP)	Hallucination (FN)
Answerable (Q_A)	False refusal (FP)	Answer attempt (TN)

A response is a hallucination when the model answers a question it has no basis for answering, hallucination(q, r) = z(q)(1 − a(r)). The hallucination rate is HR = Σ z(q)(1 − a(r)) / Σ z(q) = FN / (TP + FN). The definition is intentionally narrow: wrong answers to answerable questions are errors, but not counted as hallucinations, because some are sensible paraphrases or partial answers that overlap metrics cannot judge.

Refusal-aware correctness gives a single binary label for both kinds of query: y(q, r) = a(r) if q ∈ Q_U, and y(q, r) = (1 − a(r)) · 1[F1(r, r*) ≥ τ] if q ∈ Q_A, with τ = 0.5. A model that refuses everything would be correct on exactly the unanswerable share of the test set (17.6%), and a model that never refuses is limited by its answerable accuracy, so neither lazy strategy scores well.

4.3.3 Calibration Objective

A model is perfectly calibrated with respect to y if Pr(y = 1 | c(q) = p) = p for all p. Calibration alone is not enough, because a model that always reports its average accuracy is calibrated but useless; confidence must also discriminate, Pr(c_i > c_j | y_i = 1, y_j = 0) → 1, which is what AUROC measures. The goals are an adapter that (1) has lower ECE and Brier score, (2) has higher AUROC, (3) has a lower hallucination rate on Q_U and (4) keeps a low false-refusal rate and good answers on Q_A. None of these is optimised directly; training uses plain next-token cross-entropy over the templated sequence and only the LoRA parameters are updated. What differs between experiments is the content of the training set: how many unanswerable questions are paired with the refusal string. Goals 3 and 4 compete, since more abstention examples mean fewer hallucinations but more false refusals.

CHAPTER 5
METHODOLOGY AND TESTING

5.1 Module Description

The system is divided into six modules. Table 4 lists what each module does and which files implement it; the sections that follow describe how each one works.

Table 4. Project modules and the files that implement them.

Module	Purpose	Files
1. Data curation	Collect and clean public VIT QA rows; record sources	final_qa_dataset.csv, combined_qa_finetuning.csv, VIT_QA_Resource_Breakdown.xlsx
2. Data preparation	Build controlled-ratio splits; verify labels and leakage	prepare_expc_data.py, verify_dataset.py, expc_idk{0,10,15,20}_*.jsonl
3. Fine-tuning	QLoRA training of Normal FT and IDK FT adapters	train_lora.py, exp_c_train.py
4. Evaluation	Generate answers, confidence, refusal and calibration metrics	eval_compare_fixed.py, compare_idk_stats.py
5. Analysis and figures	Reliability, ROC, histograms, refusal and error charts	calibration_eval_utils.py, run_calibration_utils.py, make_figures.py
6. Demo chatbot	Web UI with answer, confidence and abstention	chatbot_site/

5.1.1 Module 1: Data Curation and Dataset Construction

The corpus has two sources. The first is a core QA set of 2,349 question-answer pairs built from public VIT documents. By the source inventory kept with the data, 1,930 of the pairs (82%) come from curriculum and syllabus documents and the rest from regulations, the code of conduct, the academic calendar, research admission guides, the student-life handbook and the fee structure. Questions range from direct facts (for example, the credits allotted to Professional Electives in the M.Tech in Applied Computational Fluid Dynamics) to cross-programme comparisons and student-style requests. Answers are short and factual so generated responses can be compared with the reference.

The second source is a set of 980 supplementary questions written to represent queries the documents cannot answer, in five categories: personal or record-specific queries (attendance, re-FAT status, seating); time-sensitive queries (is the FAT schedule out, library closing time today); unsupported specifics (valid details absent from the documents); non-existent entities (fake course codes); and out-of-domain queries. Rows whose source is not publicly available (for example company-specific placement-drive details; 898 rows from the knowledge file and 380 from the supplementary file) were removed before use.

5.1.2 Module 2: Data Preparation and Controlled Abstention Mixing

Abstention rows are identified with a case-insensitive lexical pattern ("don't know", "I don't", "cannot verify", "cannot see", "real-time", "not in my knowledge" and similar). It flagged 3 core pairs and 789 supplementary pairs, leaving an answerable pool of 2,537 pairs and an abstention pool of 792. The pattern is deliberately conservative, so refusals phrased differently (for example, "That is specific to your own record, and I have no way to check it") stay in the answerable pool unchanged. This is a known limitation. Every mix is shuffled with seed 42 and split 70/15/15 into train, validation and test, after checking that no question appears in both train and test.

Table 5 gives the number of examples in each split of the two training mixtures.

Table 5. Partition sizes for the two training mixtures (abstention examples in parentheses).

Mixture	ρ (target)	Total	Train	Validation	Test
IDK-0	0.00	2537	1775 (0)	380 (0)	382 (0)
IDK-15	0.15	2984	2088 (312)	447 (56)	449 (79)

Across the IDK-15 splits abstention examples make up 14.9% of train, 12.5% of validation and 17.6% of test. Random rather than stratified splitting was used so the pipelines are identical, and refusal behaviour is therefore reported separately for answerable and unanswerable queries.

For a target ratio ρ the number of abstention examples is $n_idk = \text{floor}(N_reg \cdot \rho / (1 - \rho))$, with $N_reg = 2{,}537$. For $\rho = 0.15$ this is 447 examples, sampled without replacement from the pool of 792 with seed 42. A 15% share was chosen as a compromise between enough examples to teach refusal and preserving answerable accuracy. All abstention targets are normalised to one canonical sentence: "I don't have enough information in my knowledge base to answer this question accurately." Varied refusals would be more natural, but they make refusal detection noisy and risk teaching hedging on answerable questions. The price is that refusal becomes a memorised sentence rather than an expression of measured uncertainty, which affects how confidence should be read (Section 5.1.4).

Two adapters are trained: IDK-0 (no abstention examples), called Normal FT, and IDK-15, called IDK FT, the proposed method. Answerable data, seeds, split ratios and training settings are identical; the two mixtures were split independently, so a few test questions appear in one adapter's training data (Section 5.2.3).

5.1.3 Module 3: QLoRA Fine-Tuning of Phi-3-mini

Phi-3-mini-4k-instruct is loaded in 4-bit NF4 with double quantisation and bfloat16 compute. LoRA adapters (rank 8, alpha 16, dropout 0.05) are attached to all attention and feed-forward projections so that the model can change behaviour according to content, which matters when the behaviour is a choice between answering and abstaining. Question-answer pairs use the native Phi-3 chat format, <|user|>{question}<|end|><|assistant|>{answer}<|end|>, identically in training and testing.

The settings used for both adapters are listed in Table 6; they are identical for Normal FT and IDK FT.

Table 6. Training configuration.

Setting	Value
Base model	microsoft/Phi-3-mini-4k-instruct (3.8 B parameters)
Quantisation	4-bit NF4, double quantisation, bfloat16 compute
LoRA	r = 8, alpha = 16, dropout = 0.05, bias none
Target modules	qkv_proj, o_proj, gate_up_proj, down_proj
Epochs / batch	3 epochs; batch 1 x 8 gradient-accumulation steps (effective 8)
Learning rate / optimiser	2e-4, linear decay; paged AdamW 8-bit
Max sequence length	512 tokens (longer examples truncated)
Memory settings	Gradient checkpointing on, KV cache off
Validation	10% of each training split held out for per-epoch validation loss
Seed	42 for shuffling, abstention sampling, splits and hold-out
Hardware	NVIDIA RTX 3050 Ti Laptop GPU, 4 GB VRAM

Rows whose answer begins with "NOT FOUND" are excluded from training. For IDK-15 the training run has about 235 optimiser steps per epoch.

5.1.4 Sequence-Level Confidence Estimation

Confidence is computed from the model's own token probabilities. For greedy decoding of up to 64 new tokens, $c(q) = \exp((1/T) \Sigma \log p_\theta(r_t \mid r_{<t}, q))$, the geometric mean of token probabilities, a length-normalised sequence likelihood [31]. Without normalisation a long correct answer would score lower than a one-token wrong answer. Each confidence is paired with the refusal-aware correctness label, and the teacher-forced negative log-likelihood of the reference is also recorded.

Because every abstention target is the same sentence, a model trained on it repeats the sentence with very high token probabilities. High confidence on a refusal therefore shows that the phrase was learned, not necessarily that a knowledge gap was detected, so calibration figures are read together with the refusal analysis.

5.1.5 Modules 4 and 5: Evaluation, Analysis and Figures

eval_compare_fixed.py runs the base model and the two adapters on the test file, records answer, confidence (64 new tokens), correctness, token F1, exact match and reference NLL, and exports confidences and correctness for later plotting. compare_idk_stats.py runs the refusal analysis with up to 100 new tokens and writes the refusal-rate, precision, recall, F1, false-refusal and missed-abstention figures. calibration_eval_utils.py provides AUROC and reliability-diagram functions, run_calibration_utils.py applies them to the exported per-example data, and make_figures.py draws the combined reliability diagram, ROC curves and confidence histograms.

The experiment is run in five steps. The commands assume that the scripts and the two source CSV files are in one working folder.

1. Build the two training mixtures: python prepare_expc_data.py --idk-ratio 0.0 and python prepare_expc_data.py --idk-ratio 0.15. Each run writes train, validation and test files.
2. Check the splits: python verify_dataset.py confirms that refusal rows are canonical and reports false positives of the refusal pattern.
3. Train the two adapters: python exp_c_train.py --idk-ratio 0.0 for Normal FT and python exp_c_train.py --idk-ratio 0.15 for IDK FT.
4. Evaluate calibration: python eval_compare_fixed.py runs the base model and both adapters on the IDK-15 test file and exports the per-example confidence and correctness.
5. Evaluate refusal behaviour: python compare_idk_stats.py computes the hallucination rate, IDK precision, recall and F1, and the false-refusal counts, and draws the refusal charts.

5.1.6 Module 6: Demonstration Chatbot

The Flask site offers a chat interface that returns an answer, a confidence score and the name of the backend. The default backend is a dataset lookup that blends sentence embeddings with TF-IDF and abstains with the canonical refusal sentence when nothing is close or the question names an entity that appears nowhere in the documents. A second backend loads Phi-3-mini with

a trained LoRA adapter and reports the same mean token-probability confidence used in the evaluation. The interface always labels which backend is answering.

5.2 Testing and Validation

5.2.1 Data and Pipeline Tests

Table 7 lists the checks made on the data, the metrics and the demonstration application, and their outcomes.

Table 7. Tests and validation checks.

Check	Method	Outcome
Train/test leakage	Exact-match comparison of questions between the IDK-15 train and test files	0 shared questions
Canonical refusal labels	verify_dataset.py on the IDK-15 train, validation and test files	312/312, 56/56 and 79/79 refusal rows canonical
Split sizes	Row counts of every split file against the expected sizes	Match the sizes in the partition table
Refusal detector on references	Both refusal patterns applied to the IDK-15 test references	Both flag the same 79 unanswerable questions
Lazy-baseline check	Refusal-aware correctness of a model that refuses everything, calculated from the test labels	17.6% (79 of 449), the unanswerable share
IDK-0 train vs IDK-15 test overlap	Question overlap count	15 answerable questions shared; disclosed as a limitation
Chatbot smoke test	Ask a credit question and a personal-record question in lookup mode	Credit question answered at confidence 0.80; attendance question abstained

5.2.2 Metric Definitions

- Hallucination rate HR = FN / (TP + FN); IDK precision TP / (TP + FP), recall TP / (TP + FN) and F1; false refusals; refusal rate.
- Answer quality: token-level F1 (SQuAD style), exact match, and answerable accuracy (generated answer covers more than 40% of the reference words).
- Calibration: ECE with 10 equal-width bins, Brier score, reference NLL, AUROC of confidence against refusal-aware correctness (token-F1 threshold 0.5), and reliability diagrams.

5.2.3 Experimental Setup

Three systems share base weights, 4-bit quantisation and inference prompt format. **Base**: Phi-3-mini-4k-instruct zero-shot, standard chat format, no system prompt, retrieval or demonstrations. **Normal FT**: the IDK-0 adapter, trained only on answerable questions. **IDK FT**: the IDK-15 adapter, trained on the same answerable questions plus 15% canonical abstention examples (the proposed configuration). Base versus Normal FT isolates domain adaptation; Normal FT versus IDK FT isolates the abstention data.

All three are evaluated on the IDK-15 test split of 449 queries (370 answerable, 79 unanswerable), because the IDK-0 test split has no unanswerable queries. As the two mixtures were split independently, 15 of the 449 test queries are also in the IDK-0 training set. Decoding is greedy; the refusal analysis allows 100 new tokens and the calibration analysis 64.

CHAPTER 6
PROJECT DEMONSTRATION

6.1 The Demonstration Website

The project includes a small website (Figure 6) built with Flask. It has a home page, an academics page with ready-made questions for programmes and regulations, and a campus-life page, and a chat widget that is open on every page. The styling follows the look of the public VIT website, but the site states clearly that it is a student project and not an official VIT site.

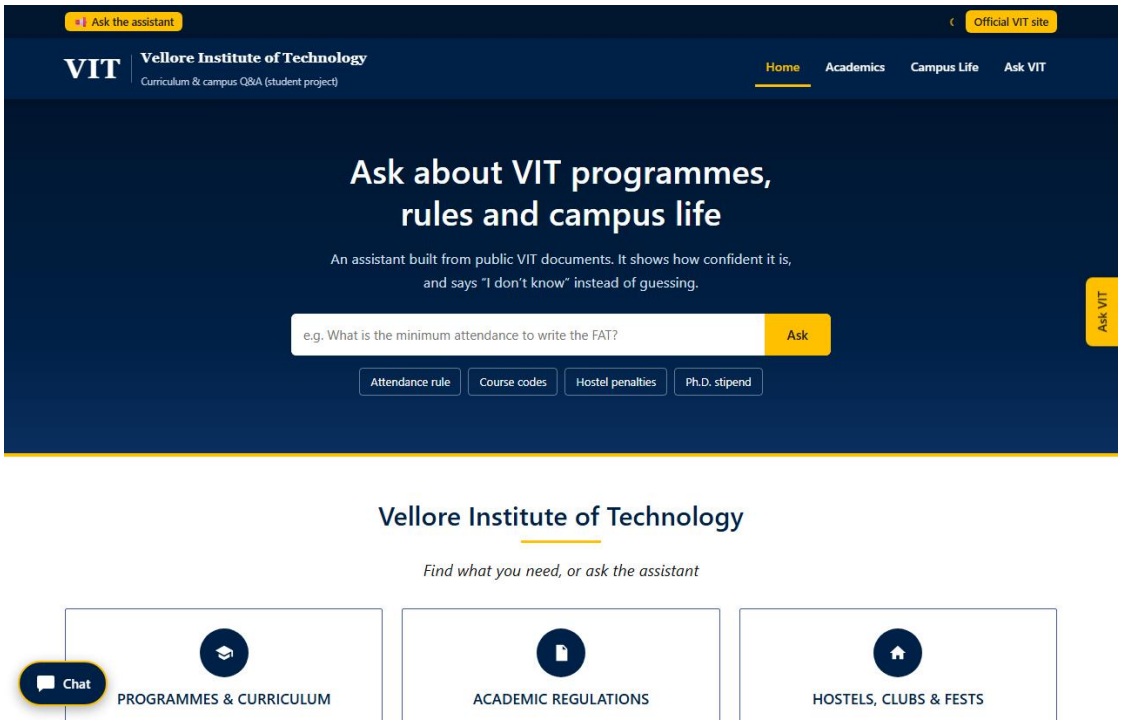


Figure 6. Home page of the demonstration website.

6.2 Example Interaction

Figure 7 shows two questions asked in the chat widget. The first, about the total credit requirement of the M.Sc. Biotechnology programme, is answered from the stored question-answer pairs with a confidence bar. The second asks about a student's own attendance today, which no document can answer, so the assistant gives the canonical refusal and marks it as "Not sure". The widget header names the backend that produced the answers.

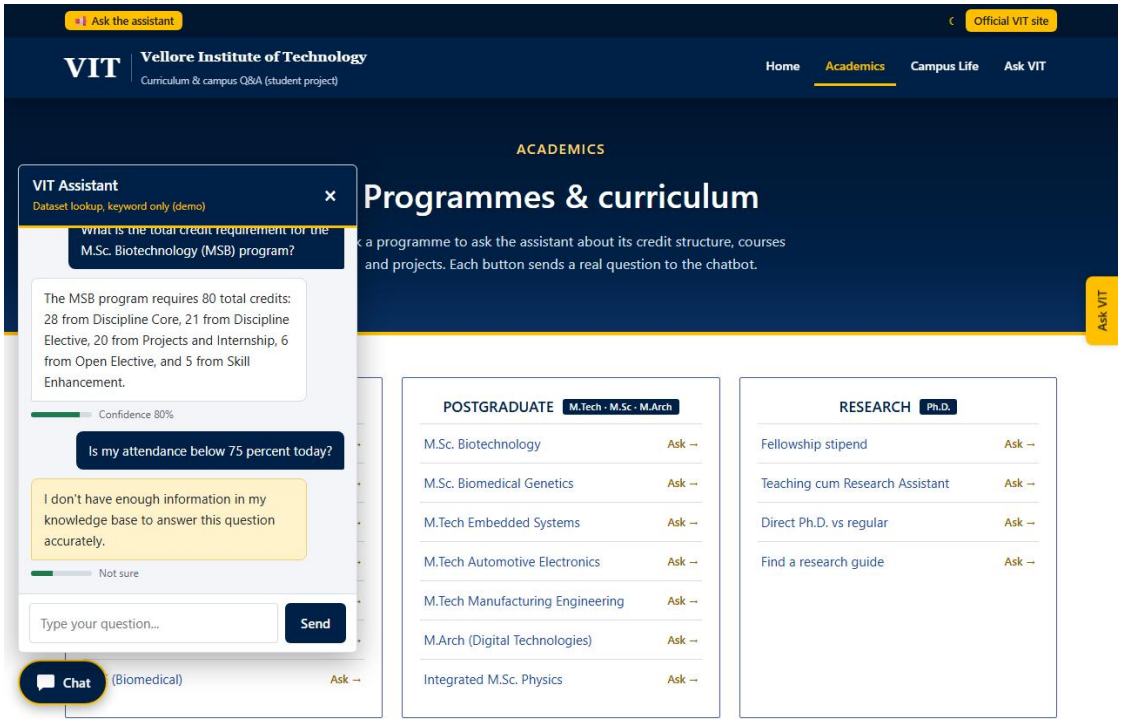


Figure 7. Chat widget answering one question and abstaining on another.

The screenshots were taken in the default lookup mode, which needs no GPU. In this mode the confidence is the retrieval score, not the token probability used in the experiments. A second mode loads Phi-3-mini with a trained LoRA adapter and shows the same mean token-probability confidence as the evaluation; it needs the adapter files and a GPU.

6.3 Running the Demonstration

1. Install the requirements: pip install -r chatbot_site/requirements.txt.
2. Start the site: python chatbot_site/app.py and open http://127.0.0.1:5000.
3. For the fine-tuned model, set CHATBOT_BACKEND=lora and ADAPTER_PATH to the adapter folder before starting.

CHAPTER 7
RESULT AND DISCUSSION

7.1 Hallucination Reduction

Table 8 reports the refusal behaviour of the three systems on the 449 test queries, and Figures 8 and 9 show the same results graphically.

Table 8. Hallucinations and refusals on the IDK-15 test set (79 unanswerable, 370 answerable).

System	Refusal rate	IDK precision	IDK recall	IDK F1	HR	Missed abst. (FN)	False refusals (FP)	Answerable acc.
Base	0.345	0.252	0.494	0.333	0.506	40	116	0.227
Normal FT	0.000	0.000	0.000	0.000	1.000	79	0	0.451
IDK FT	0.185	0.819	0.861	0.840	0.139	11	15	0.408

Before fine-tuning the base model refuses about half of the unanswerable questions (HR = 0.506). After fine-tuning on answerable data only, Normal FT refuses none of the 449 test questions: it answers all 79 unanswerable ones, including questions about individual student records, live schedules and non-existent courses. Domain adaptation lifted answerable accuracy from 0.227 to 0.451 while teaching the model to answer everything. On hallucination, fine-tuning was worse than no fine-tuning.

With 15% abstention data, IDK FT refuses 68 of the 79 unanswerable queries. HR falls to 0.139, a relative reduction of 86% against Normal FT and 73% against the base model. Of its 83 refusals, 68 are correct (precision 0.819), giving an IDK F1 of 0.840. Its refusal rate (18.5%) is close to the true share of unanswerable queries (17.6%). The base model's recall of 0.494 is not knowledge-awareness: 116 of its 155 refusals are on answerable queries, mostly generic statements about access to institutional information.

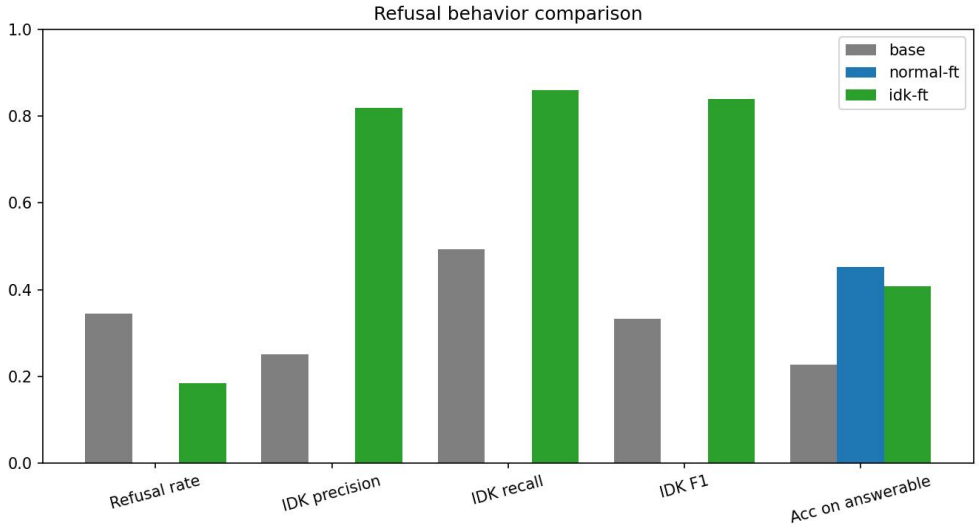


Figure 8. Refusal behaviour of Base, Normal FT and IDK FT.

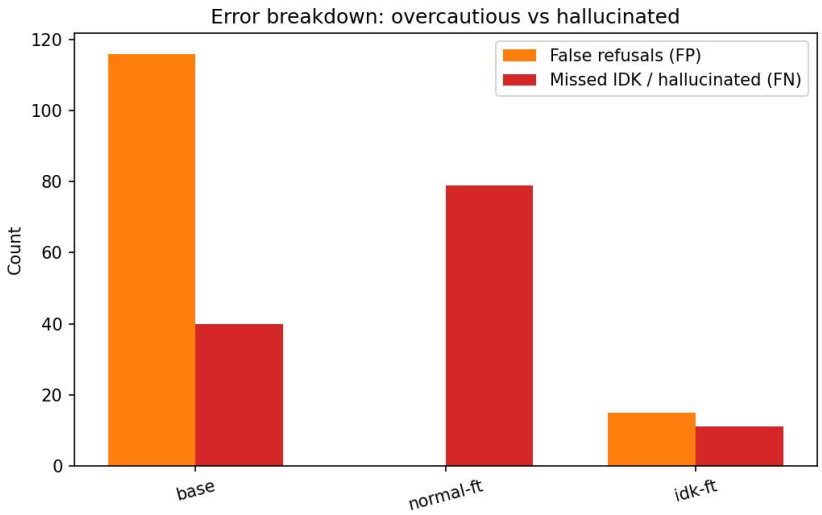


Figure 9. False refusals (over-caution) and missed abstentions (hallucinations) per system.

7.2 Calibration Improvement

Table 9 reports calibration and answer quality. Figure 10 shows the reliability diagram, which reveals the direction of the calibration error that ECE alone does not show.

Table 9. Calibration and answer quality on the IDK-15 test set (refusal-aware correctness, token-F1 threshold 0.5).

Metric	Base	Normal FT	IDK FT
Accuracy (refusal-aware)	0.100	0.327	0.477
Exact match	0.000	0.051	0.189
Token F1	0.168	0.391	0.515
ECE	0.622	0.408	0.304
Brier score	0.480	0.351	0.285
Reference NLL	2.405	1.663	0.965
AUROC	0.557	0.764	0.810

ECE falls from 0.622 (Base) to 0.408 (Normal FT) to 0.304 (IDK FT), a 51% reduction in total, of which about 25% comes from abstention data. The Brier score follows (0.480, 0.351, 0.285), so the improvement is not a binning artefact. All three reliability curves lie below the diagonal, so all systems are over-confident. The Base curve is nearly flat; Normal FT rises but plateaus near 0.55 accuracy above confidence 0.8; IDK FT rises monotonically and reaches about 0.80 accuracy at a mean confidence of about 0.97, but at a confidence near 0.65 it is right only 19% of the time.

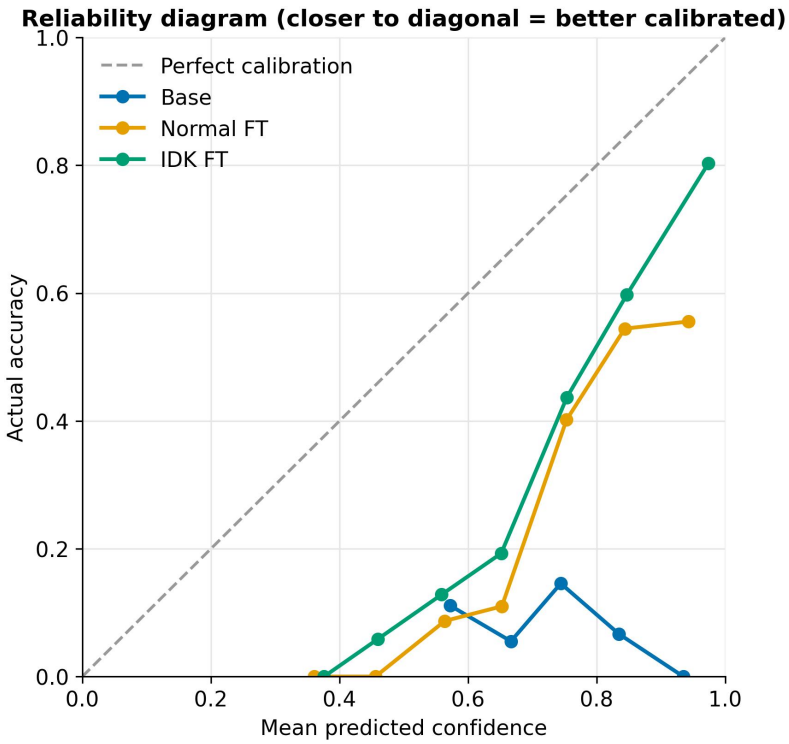


Figure 10. Reliability diagram for Base, Normal FT and IDK FT (10 equal-width bins); the dashed line is perfect calibration.

Two caveats apply. First, much of the gain comes from unanswerable questions: IDK FT repeats the canonical refusal exactly, so correct abstentions get near-perfect token F1 and high confidence. Exact matches rise from 23 to 85 of 449, mostly from these. Second, the rise in refusal-aware accuracy (0.327 to 0.477, about 67 more correct responses) roughly equals the number of unanswerable questions now answered correctly; on answerable questions token F1 is about 0.39 for IDK FT against 0.40 for Normal FT.

7.3 Confidence Separation

For the base model the confidence distributions of correct and incorrect answers overlap almost completely (roughly 0.55 to 0.85) and AUROC is 0.557. Fine-tuning separates them: for Normal FT correct answers move to 0.7-1.0 while incorrect ones sit mostly at 0.55-0.75, with a tail up to 0.95 that includes confident answers to unanswerable questions (AUROC 0.764). IDK FT is best (AUROC 0.810), with a peak of over 80 correct answers in the 0.95-1.0 bin, mostly canonical refusals. Its ROC curve reaches about 0.28 true-positive rate at almost zero false-positive rate, against about 0.05 for Normal FT, and at a false-positive rate of 0.1 it keeps about 43% of correct answers against 25%. The two curves converge above a false-positive rate of about 0.4, so the gain lies in the high-confidence region where a practical threshold would sit. The same peak holds about 15 false positives, matching the 15 false refusals: a fixed refusal sentence can be produced with high confidence even when refusing is wrong.

Figure 11 shows how the confidence of correct and incorrect responses is distributed, and Figure 12 shows the corresponding ROC curves.

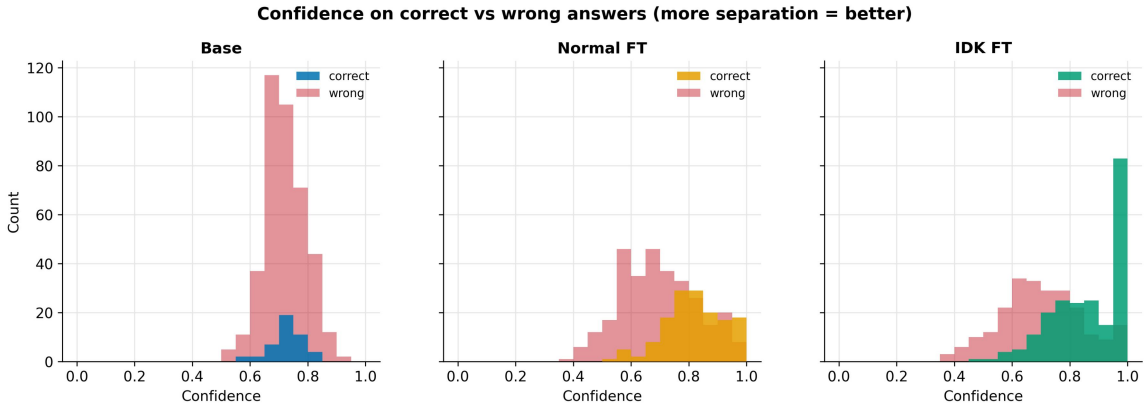


Figure 11. Confidence on correct and incorrect responses for each system.

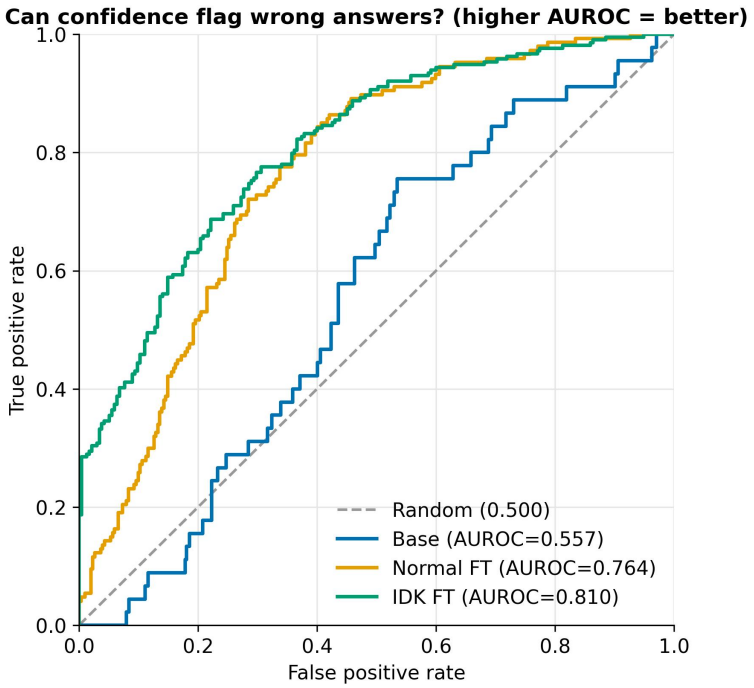


Figure 12. ROC curves for separating correct from incorrect responses using sequence-level confidence.

7.4 Cost of Abstention: False Refusals

IDK FT refuses 15 of 370 answerable queries (false refusal rate 4.1%), against 31.4% for the base model (116 refusals) and 0% for Normal FT. Answerable accuracy falls from 0.451 to 0.408 (4.3 points, 167 to 151 correct answers). The 16 correct answers lost are about the number of false refusals, so the penalty comes from refusals rather than poorer answers: IDK FT is right on 151 of the 355 answerable queries it attempts (42.5%), against 167 of 370 (45.1%) for Normal FT. For 16 fewer correct answers, IDK FT avoids 68 hallucinations, roughly four per correct answer given up. For a student assistant, a wrong credit or deadline is costlier than an unnecessary pointer to VTOP or the proctor. As 15 answerable test questions are in the training data of Normal FT only, its accuracy is slightly overstated and the true gap may be below 4.3 points.

7.5 Contribution of Each Training Stage

Because Normal FT and IDK FT differ only in the abstention data, and Base and Normal FT differ only in domain adaptation, the contribution of each stage can be separated (Table 10).

Table 10. Changes attributable to each stage.

Metric	Domain adaptation (Base → Normal FT)	Abstention data (Normal FT → IDK FT)
Hallucination rate	+0.494 (0.506 → 1.000)	−0.861 (1.000 → 0.139)
IDK F1	−0.333	+0.840
False refusal rate	−0.314	+0.041
Answerable accuracy	+0.224	−0.043
Token F1	+0.223	+0.124
ECE	−0.214	−0.104
Brier	−0.129	−0.066
AUROC	+0.207	+0.046

Domain adaptation gives the main gain in knowledge: it roughly doubles answerable accuracy, removes the base model's indiscriminate refusals and gives the largest single improvements in ECE, Brier score and AUROC, but it removes abstention altogether. The abstention data repairs exactly that, taking HR from 1.000 to 0.139, and adds further calibration and discrimination gains, at the cost of 4.1% false refusals and 4.3 points of answerable accuracy.

7.6 Qualitative Error Analysis

1. Missed abstentions (11): questions that look answerable, for example valid-looking but non-existent course codes for which the model invents titles and credits, or details absent from every document. These are the most dangerous errors because they sound correct; a medium confidence value could act as a threshold signal.
2. False refusals (15 of 370): possibly triggered by wording that appears in refusal examples ("my", "attendance", "schedule", "today") or by programmes the model rarely saw. They are produced with confidence above 0.95, in the same region as correct refusals.
3. Wrong answers to answerable questions: the largest group. Typical errors are wrong numbers (credits, course codes) and repeating the question without facts. Abstention training does not affect them; they reflect the limited memorisation of a 3.8B model on about 1,800 training examples.

7.7 Answers to the Research Questions

1. **Does supervised abstention reduce hallucination?** Yes. Against a model fine-tuned only on answerable data, the hallucination rate falls from 1.000 to 0.139 (68 of the 79 unanswerable queries are refused), and the IDK F1 rises from 0.000 to 0.840.
2. **Does it improve confidence calibration?** Yes, within the limits discussed above. ECE falls from 0.408 to 0.304, the Brier score from 0.351 to 0.285 and AUROC rises from 0.764 to 0.810. Part of this gain comes from the confident repetition of the fixed refusal sentence, and every system remains over-confident.
3. **What is the cost to answerable questions?** Small but real. IDK FT refuses 15 of 370 answerable queries (4.1%) and its answerable accuracy is 4.3 points lower (0.408 against 0.451); the loss comes from the refusals, not from worse answers.

7.8 Cost Analysis

Table 11 sets the cost of abstention training against its benefit, using the figures reported above.

Table 11. Cost and benefit of abstention training.

Item	Normal FT	IDK FT
Training rows	1,775	2,088 (+313, about 18%)
Training schedule	3 epochs, effective batch 8	3 epochs, effective batch 8
Paid services or cloud compute	None	None
Correct answerable responses (of 370)	167	151 (−16)
False refusals	0	15
Unanswerable queries answered (hallucinations)	79	11 (−68)

The extra training data adds about 18% to the number of examples and no extra schedule, no paid service and no extra hardware. The price in answer quality is 16 correct answerable responses, and in return 68 fabricated answers to unanswerable questions are avoided.

7.9 Discussion and Limitations

The central finding is that abstention does not emerge from fine-tuning alone: standard fine-tuning answered all 79 unanswerable questions, while 15% explicit refusal data reduced hallucination to 0.139. Abstention training changed how the model behaves on unanswerable questions; it did not improve the quality of answers on answerable ones or remove over-confidence. The limitations are:

- Single model, single ratio (15%), single seed and one institution; 15% is not shown to be optimal. HR = 0.139 is 11 of 79 queries, a 95% interval of about 0.08 to 0.23.
- Refusals are detected with a phrase list and training uses one canonical sentence, so the model may learn a script, and some refusal-style supplementary rows with different wording remain in the answerable pool.
- Correctness uses token overlap, which does not check whether numbers or course codes are right and penalises valid paraphrases; accuracy values are for comparison, not absolute.
- Unanswerable questions were designed by category rather than collected from real students, and the split is random, so test questions may be close to training questions.
- Confidence is the mean token probability with no calibration step such as temperature scaling, and the model answers from memory without retrieval.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project examined whether the composition of fine-tuning data alone can improve the calibration and reduce the hallucination of a small language model. Phi-3-mini was fine-tuned with QLoRA under a 4 GB VRAM limit for VIT questions and three systems were compared: the base model, a domain fine-tuned model and an abstention-aware model with 15% canonical refusal examples.

1. Domain fine-tuning alone is not enough. It about doubled answerable accuracy (0.227 to 0.451) but removed abstention: Normal FT answered all 79 unanswerable queries, raising HR from 0.506 to 1.000.
2. Controlled abstention data cuts hallucination. With 15% abstention examples HR fell to 0.139 and IDK F1 reached 0.840, at the cost of 15 false refusals out of 370 answerable queries (4.1%) and a 4.3-point drop in answerable accuracy.
3. The same change improves calibration and discrimination without any calibration objective: ECE 0.622 → 0.408 → 0.304, Brier 0.480 → 0.351 → 0.285 and AUROC 0.557 → 0.764 → 0.810.

The findings are bounded by over-confidence in all systems, partly by the model repeating one fixed refusal sentence at high confidence, and by a single seed, ratio, model and institution.

8.2 Future Work

- Abstention-ratio sweep (the 10% and 20% splits already exist in the project) with several seeds, to find the best ratio.
- Context-dependent refusals (for example pointing the user to VTOP or the proctor) against one canonical sentence, to see whether refusal can express uncertainty rather than a memorised phrase.
- Post-hoc calibration such as temperature scaling on the validation split, and selective prediction with a chosen confidence threshold.
- Other uncertainty signals: sampling-based semantic agreement, verbalised confidence and probes of internal states.
- Better correctness measurement with NLI, LLM judges, human review and explicit checks of course codes and credit values.
- Combining the abstention-trained model with retrieval from the curriculum documents.
- Other sub-4B models (Llama, Qwen, Gemma) and other institutions.

CHAPTER 9
REFERENCES

[1] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," arXiv:2404.14219, 2024.

[2] D. Biderman et al., "LoRA learns less and forgets less," Trans. Mach. Learn. Res., 2024.

[3] F. Brahman et al., "The art of saying no: Contextual noncompliance in language models," in Proc. NeurIPS Datasets and Benchmarks Track, 2024.

[4] G. W. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1-3, 1950.

[5] Q. Cheng et al., "Can AI assistants know what they don't know?" in Proc. ICML, 2024.

[6] C. K. Chow, "On optimum recognition error and reject tradeoff," IEEE Trans. Inf. Theory, vol. 16, no. 1, pp. 41-46, 1970.

[7] T. Dettmers, M. Lewis, S. Shleifer and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," in Proc. ICLR, 2022.

[8] T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Proc. NeurIPS, 2023.

[9] R. El-Yaniv and Y. Wiener, "On the foundations of noise-free selective classification," J. Mach. Learn. Res., vol. 11, pp. 1605-1641, 2010.

[10] S. Feng et al., "Don't hallucinate, abstain: Identifying LLM knowledge gaps via multi-LLM collaboration," in Proc. ACL, 2024.

[11] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in Proc. NeurIPS, 2017.

[12] Z. Gekhman et al., "Does fine-tuning LLMs on new knowledge encourage hallucinations?" in Proc. EMNLP, 2024.

[13] S. Gunasekar et al., "Textbooks are all you need," arXiv:2306.11644, 2023.

[14] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. ICML, PMLR vol. 70, pp. 1321-1330, 2017.

[15] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in Proc. ICML, PMLR vol. 97, pp. 2790-2799, 2019.

[16] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in Proc. ICLR, 2022.

[17] X. Hu et al., "Small updates, big doubts: Does parameter-efficient fine-tuning enhance hallucination detection?" arXiv:2602.11166, 2026.

[18] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," ACM Trans. Inf. Syst., vol. 43, no. 2, 2025.

[19] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, pp. 1-38, 2023.

[20] Z. Jiang, J. Araki, H. Ding and G. Neubig, "How can we know when language models know? On the calibration of language models for question answering," Trans. Assoc. Comput. Linguist., vol. 9, pp. 962-977, 2021.

[21] S. Kadavath et al., "Language models (mostly) know what they know," arXiv:2207.05221, 2022.

[22] A. T. Kalai, O. Nachum, S. S. Vempala and E. Zhang, "Why language models hallucinate," arXiv:2509.04664, 2025.

[23] A. T. Kalai and S. S. Vempala, "Calibrated language models must hallucinate," in Proc. ACM STOC, 2024.

[24] A. Kamath, R. Jia and P. Liang, "Selective question answering under domain shift," in Proc. ACL, 2020.

[25] S. Kapoor et al., "Large language models must be taught to know what they don't know," in Proc. NeurIPS, 2024.

[26] L. Kuhn, Y. Gal and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in Proc. ICLR, 2023.

[27] B. Lester, R. Al-Rfou and N. Constant, "The power of scale for parameter-efficient prompt tuning," in Proc. EMNLP, 2021.

[28] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.

[29] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in Proc. ACL, pp. 4582-4597, 2021.

[30] S. Lin, J. Hilton and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in Proc. ACL, pp. 3214-3252, 2022.

[31] A. Malinin and M. Gales, "Uncertainty estimation in autoregressive structured prediction," in Proc. ICLR, 2021.

[32] P. Manakul, A. Liusie and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in Proc. EMNLP, 2023.

[33] S. Mangrulkar et al., "PEFT: State-of-the-art parameter-efficient fine-tuning methods," https://github.com/huggingface/peft, 2022.

[34] M. P. Naeini, G. F. Cooper and M. Hauskrecht, "Obtaining well calibrated probabilities using Bayesian binning," in Proc. AAAI, vol. 29, 2015.

[35] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.

[36] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. NeurIPS, 2019.

[37] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[38] S. Porwal, S. Prasath S, X. Bi and M. Scandlen, "Evaluating hallucinations in domain-adapted large language models," arXiv:2606.07521, 2026.

[39] P. Rajpurkar, R. Jia and P. Liang, "Know what you don't know: Unanswerable questions for SQuAD," in Proc. ACL, pp. 784-789, 2018.

[40] P. Rajpurkar, J. Zhang, K. Lopyrev and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in Proc. EMNLP, pp. 2383-2392, 2016.

[41] K. Tian et al., "Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback," in Proc. EMNLP, 2023.

[42] B. Wen et al., "Know your limits: A survey of abstention in large language models," Trans. Assoc. Comput. Linguist., 2025.

[43] L. von Werra et al., "TRL: Transformer reinforcement learning," https://github.com/huggingface/trl, 2020.

[44] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. EMNLP System Demonstrations, pp. 38-45, 2020.

[45] M. Xiong et al., "Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs," in Proc. ICLR, 2024.

[46] Y. Yang, E. Chern, X. Qiu, G. Neubig and P. Liu, "Alignment for honesty," in Proc. NeurIPS, 2024.

[47] C. Zhang et al., "Confidence-calibrated small-large language model collaboration for cost-efficient reasoning," arXiv:2603.03752, 2026.

[48] H. Zhang et al., "R-Tuning: Instructing large language models to say 'I don't know'," in Proc. NAACL, 2024.

APPENDIX A
KEY CODE EXCERPTS

The complete source of every script is kept in the code folder of the project bundle. The excerpts below show the parts that carry the main ideas of the method: controlled abstention mixing, the QLoRA configuration, the confidence score with refusal-aware correctness, and the calibration error.

A.1 Controlled abstention mixing (prepare_expc_data.py)

```
n_idk_needed = int(n_regular * idk_ratio / (1 - idk_ratio))
idk_pool["answer"] = IDK_ANSWER            # one canonical refusal string
idk_sample = idk_pool.sample(n=n_idk_needed, random_state=42)
full_df = pd.concat([regular_rows, idk_sample]).sample(frac=1, random_state=42)
# 70 / 15 / 15 split into train, valid and test
```

A.2 4-bit and LoRA configuration (train_lora.py)

```
BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                   bnb_4bit_compute_dtype=torch.bfloat16,
bnb_4bit_use_double_quant=True)
LoraConfig(r=8, lora_alpha=16, lora_dropout=0.05, bias="none",
task_type="CAUSAL_LM",
           target_modules=["qkv_proj", "o_proj", "gate_up_proj", "down_proj"])
```

A.3 Confidence and refusal-aware correctness (eval_compare_fixed.py)

```
logprobs = [F.log_softmax(s[0], dim=-1)[t].item() for s, t in zip(gen.scores,
new_tokens)]
conf = float(torch.tensor(logprobs).mean().exp())     # geometric mean of token
probabilities

def is_correct(pred, ref, is_idk):
    if is_idk:           return int(refused(pred))
    if refused(pred):    return 0
    return int(token_f1(pred, ref) >= F1_THRESHOLD)   # 0.5
```

A.4 Expected Calibration Error (eval_compare_fixed.py)

```
for b in bins:
    if b:
        avg_conf = sum(r["confidence"] for r in b) / len(b)
        avg_acc  = sum(r["correct"] for r in b) / len(b)
        score += len(b) / total * abs(avg_conf - avg_acc)
```